Recognizing a specific object like a 'watch' in an image generally requires more sophisticated techniques than simple filters or morphological operations. It often involves:

1. **Feature Detection and Matching:** Identifying unique points or patterns (features) in objects and comparing them across images.
2. **Template Matching:** Searching for a small template image within a larger image.
3. **Machine Learning/Deep Learning Models:** Training a model on a large dataset of watch images to learn their characteristics. (This is the most robust but also the most complex approach).

For this demonstration, we'll use **Feature Detection and Matching with ORB (Oriented FAST and Rotated BRIEF)**, a fast and efficient algorithm available in OpenCV. This method works by finding distinctive keypoints in both a 'query' image (the watch template) and a 'target' image (our main image) and then trying to match them.

Important Note: For successful detection, a good quality template image of the watch you want to find is crucial. Since we don't have a specific watch image, we'll use a placeholder for the `watch_template.jpg`. You would replace this with your actual watch image file.

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

# PROGRAM: Feature Detection and Matching for Object Recognition

# Our main input image (V KOHLI.jpeg) is already loaded into 'input_image' from previous cells.
# We will use 'input_image' as our target image.
target_image = input_image.copy()

# --- Step 1: Create a dummy watch template image for demonstration ---
# In a real scenario, you would load an actual image of a watch here.
# Example: watch_template = cv2.imread('path/to/your/watch_image.jpg', cv2.IMREAD_GRAYSCALE)

# Creating a simple white square as a dummy template for code execution without errors
# This will NOT find a watch in the image, but demonstrates the process.
dummy_template_size = 100
```

```
dummy_watch_template_color = np.zeros((dummy_template_size, dummy_template_size, 3), dtype=np.uint8)
dummy_watch_template_color[:, :, :] = 255 # Make it white
cv2.imwrite("watch_template.jpg", dummy_watch_template_color)

# Load the dummy template (or your actual watch template)
watch_template = cv2.imread("watch_template.jpg", cv2.IMREAD_GRAYSCALE)

if watch_template is None:
    print("Error: Could not load watch_template.jpg. Please ensure it exists and is valid.")
else:
    # Convert target_image to grayscale for ORB processing
    gray_target_image = cv2.cvtColor(target_image, cv2.COLOR_BGR2GRAY)

    # --- Step 2: Initialize the ORB detector ---
    orb = cv2.ORB_create(nfeatures=500) # You can adjust nfeatures

    # --- Step 3: Find the keypoints and descriptors with ORB ---
    kp1, des1 = orb.detectAndCompute(watch_template, None) # Keypoints and descriptors for template
    kp2, des2 = orb.detectAndCompute(gray_target_image, None) # Keypoints and descriptors for target

    # --- Step 4: Create a Brute-Force Matcher object ---
    # It takes two arguments: crossCheck (boolean) and normType.
    # For ORB, use NORM_HAMMING because it uses binary descriptors.
    bf = cv2.BFMatcher(cv2.NORM_HAMMING, crossCheck=True)

    # --- Step 5: Match descriptors ---
    matches = bf.match(des1, des2)

    # --- Step 6: Sort them in the order of their distance ---
    # Less distance means better match.
    matches = sorted(matches, key=lambda x: x.distance)

    # --- Step 7: Draw first 10 matches ---
    # You can adjust the number of matches drawn.
    img_matches = cv2.drawMatches(watch_template, kp1, target_image, kp2, matches[:10], None, flags=c

    # --- Step 8: Display the result ---
    plt.figure(figsize=(15, 7))
```

```
    plt.imshow(cv2.cvtColor(img_matches, cv2.COLOR_BGR2RGB))
    plt.title('ORB Feature Matching: Watch Template vs. Target Image')
    plt.axis('off')
    plt.show()

    print("Feature matching complete. Note: This demonstration uses a dummy template. Replace 'watch_

# cv2.imshow and cv2.waitKey do not work in Google Colab
# cv2.destroyAllWindows() also not needed in Colab for display
```


ORB Feature Matching: Watch Template vs. Target Image
Virat Kohli
MRF